CareBot AI - Complete Project Documentation

An educational, full-stack AI healthcare assistant that combines classical machine learning, rule-based NLP, and computer-vision heuristics to provide *preliminary* health assessments, guidance, and education.

> [!] **CareBot AI is NOT medical software.** It is a portfolio / educational project. Every prediction, risk level, and recommendation is AI-generated and is **not a medical diagnosis**. It must never be used for real medical decisions. Always consult a licensed healthcare professional.

Table of Contents

1. System Overview
2. Architecture
3. Technology Stack - What & Why
4. Module 1 - Disease Prediction (Machine Learning)
5. The Dataset
6. Module 2 - AI Chatbot (Rule-Based NLP)
7. Module 3 - Image Detection (Heuristic, NOT a CNN)
8. Module 4 - Recommendations, Risk & Hospitals
9. Supporting Modules
10. Why These Choices Over Alternatives
11. Limitations & Honesty
12. How to Run & Retrain

---

1. System Overview

CareBot AI is a **client-server web application**:

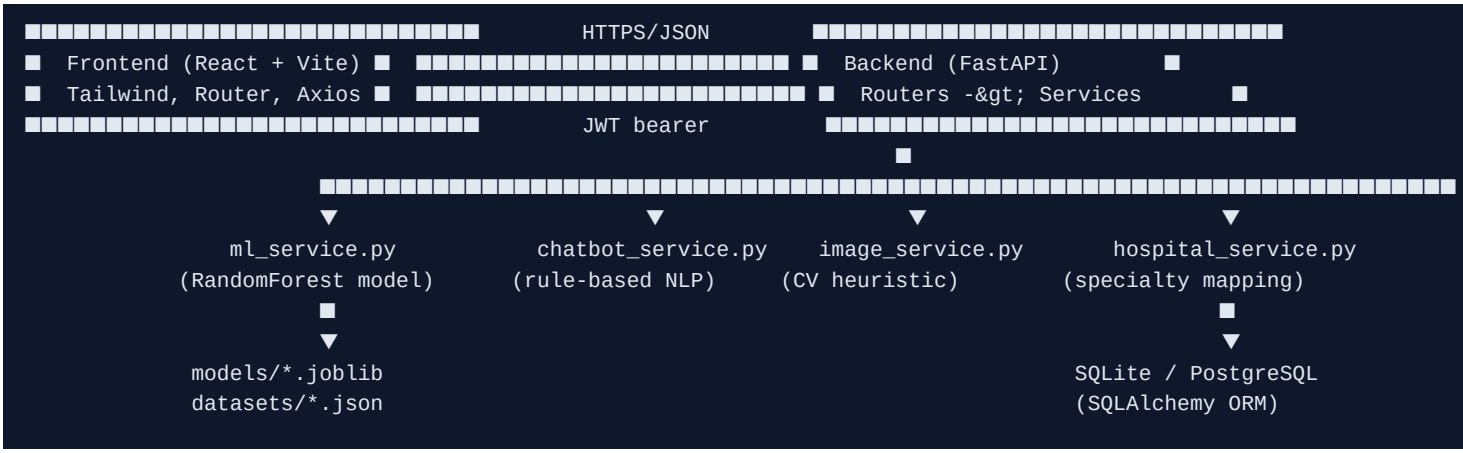
- A **React single-page app** (the user interface).
- A **FastAPI backend** (REST API, auth, business logic).
- A **trained machine-learning model** for disease prediction.
- A **SQLite/PostgreSQL database** for users, history, and uploads.

Module	Technique	Status
Disease prediction	RandomForest (scikit-learn)	[OK] Fully working ML model
AI chatbot	Rule-based NLP (no LLM)	[OK] Fully working

Image detection	**Computer-vision heuristic (NOT a CNN)**	[!] Heuristic stand-in
Recommendations	Curated knowledge base	[OK] Fully working
Risk assessment	Confidence × severity rules	[OK] Fully working
Hospital locator	Disease->specialty + real Maps links	[!] Mock names, real links
Auth / Dashboard / Reports	JWT + SQLAlchemy + Recharts	[OK] Fully working

---

2. Architecture



Design pattern: thin **routers** (HTTP layer) delegate to **services** (business logic), which load **artifacts** (the trained model, JSON knowledge base). This separation makes each piece independently testable and swappable - e.g. the heuristic image service can be replaced by a CNN without touching the router or the frontend.

---

3. Technology Stack - What & Why

3.1 Frontend

Technology	Role	Why this, not the alternative
React	UI library	Component model fits a multi-page dashboard; huge ecosystem. Chosen over **Angular** (heavier, opinionated) and **vanilla JS** (unmaintainable at this size).

Vite	Build tool / dev server	Near-instant hot-reload and fast builds via native ESM. Chosen over **Create React App** (deprecated, slow Webpack) and **Next.js** (SSR/routing we don't need for a pure SPA).
Tailwind CSS	Styling	Utility-first = consistent design without context-switching to CSS files. Chosen over **Bootstrap** (generic look, heavier) and **plain CSS** (repetitive, harder to keep consistent).
React Router	Client-side routing	Standard for SPA navigation between pages (Predictor, Assistant, Dashboard...).
Axios	HTTP client	Interceptors let us attach the JWT and handle 401->logout globally in one place. Chosen over raw **fetch** (no interceptors, more boilerplate).
Recharts	Charts (dashboard)	Declarative React charts for the history/analytics dashboard. Chosen over **Chart.js** (imperative, canvas) for cleaner React integration.
Framer Motion	Animations	Simple, physics-based transitions for a polished feel.
Lucide Icons	Iconography	Lightweight, tree-shakeable SVG icon set.

3.2 Backend

Technology	Role	Why this, not the alternative
FastAPI	Web framework / REST API	Async, extremely fast (Starlette), **automatic OpenAPI docs** at `/docs`, and Pydantic validation built in. Chosen over **Flask** (no built-in validation or async, no auto docs) and **Django** (heavyweight, ORM/admin we don't need).
Uvicorn	ASGI server	High-performance ASGI server that runs FastAPI's async app.
Pydantic	Data validation / schemas	Declaratively validates every request/response; guarantees the API contract. Eliminates a whole class of bugs.

SQLAlchemy	ORM / database layer	Lets the same code target **SQLite in dev and PostgreSQL in prod** by only changing a connection string. Chosen over raw SQL (unsafe, verbose) and Django ORM (tied to Django).
python-jose	JWT tokens	Stateless authentication - no server-side session store needed; scales horizontally. Chosen over server sessions (stateful, harder to scale).
passlib + bcrypt	Password hashing	bcrypt is a slow, salted, industry-standard hash - safe against brute force. Never store plaintext or fast hashes (MD5/SHA).

3.3 Machine Learning & Data

Technology	Role	Why this, not the alternative
scikit-learn	ML model (RandomForest)	Best-in-class for **classical tabular ML**; simple, reliable, no GPU. Chosen over TensorFlow/PyTorch (deep learning is overkill for a 134-feature tabular problem - see §4).
pandas	Data handling	Tabular manipulation of the symptom-disease matrix.
NumPy	Numerical arrays	Fast vectorized math for feature vectors and the image heuristic.
joblib	Model persistence	Efficiently serializes the trained scikit-learn model to disk (`.joblib`).
Pillow (PIL)	Image processing	Decoding/resizing uploaded images for the heuristic analysis.

3.4 Database & Infrastructure

Technology	Role	Why
SQLite	Dev database	Zero-config, file-based - perfect for local development and demos.
PostgreSQL	Prod database (optional)	Robust, concurrent, production-grade - SQLAlchemy makes the switch a one-line change.

Docker + docker-compose	Containerization	Reproducible "runs the same everywhere" deployment for both services.

---

4. Module 1 - Disease Prediction (Machine Learning)

This is the **core ML module** and the most important technical decision in the project.

4.1 The Problem

Given a set of symptoms (from a fixed vocabulary of **134 symptoms**), predict the most probable **condition(s)** out of **40 diseases**, with a confidence score and a risk level. It's a **multi-class classification** problem on **binary tabular features** (symptom present = 1, absent = 0).

4.2 The Algorithm - Random Forest Classifier

We use a `RandomForestClassifier` from scikit-learn.

What a Random Forest is (plainly): it builds many independent **decision trees**, each trained on a random subset of the data and a random subset of the features. To predict, every tree "votes," and the forest averages their probabilities. This *ensemble* approach cancels out the overfitting that a single decision tree suffers from.

Exact configuration (`scripts/train_model.py`):

```
RandomForestClassifier(
   n_estimators=400,          # 400 trees  -  more trees = more stable votes
   max_depth=None,            # let trees grow fully
   min_samples_leaf=1,
   max_features="sqrt",       # each split considers sqrt134 ~ 12 features -&gt; decorrelates trees
   class_weight="balanced",   # every disease weighted equally regardless of row count
   random_state=42,           # reproducible results
   n_jobs=-1,                 # train on all CPU cores
)
```

4.3 Why Random Forest - and not the alternatives

Candidate	Why we did not choose it
Deep Neural Network (CNN/MLP)	Needs large data + GPU + tuning. On a small, clean, **tabular** dataset it overfits and offers no benefit. Deep learning shines on images/text/audio, **not** 134 binary columns.
Logistic Regression	Linear - can't capture symptom **interactions** (e.g. "fever AND rash together" implies something different than either alone). Random Forest captures these automatically.
Single Decision Tree	High variance - memorizes the training data and generalizes poorly. The forest fixes exactly this.

SVM	Doesn't natively output well-calibrated multi-class probabilities; slower to tune for 40 classes.
XGBoost / Gradient Boosting	Excellent, but heavier dependency and more tuning for marginal gain here. Random Forest already scores ~99% and is simpler and more interpretable.
Naive Bayes	Assumes symptoms are independent - false in medicine (symptoms strongly co-occur), which hurts accuracy.

Random Forest is the sweet spot: it captures feature interactions, resists overfitting, needs no GPU, gives probability estimates (needed for the top-3 + confidence UI), is fast to train, and is reasonably interpretable (feature importances).

4.4 Training Process

1. `scripts/generate_dataset.py` builds the symptom-disease matrix (see §5).
2. `scripts/train_model.py` loads it, label-encodes the 40 disease names, and does a stratified **80/20 train/test split** (4,800 train / 1,200 test).
3. Fits the Random Forest, evaluates, and saves artifacts with joblib: - `models/disease_model.joblib` - the trained forest - `models/label_encoder.joblib` - maps class indices <-> disease names - `models/symptoms_list.json` - the ordered 134-feature vocabulary - `models/model_metrics.json` - evaluation results

4.5 Evaluation - and why these metrics

Metric	Value	Why it matters
Top-1 accuracy	**99.58%**	The single best guess is correct on the held-out test set.
Top-3 accuracy	**100.00%**	The true disease is in the top-3 every time - and the UI **shows the top 3**, so this is the metric that actually matches the product.
5-fold Cross-Validation	**99.55% ± 0.20%**	Trains/tests on 5 different splits - an **honest, less optimistic** estimate that guards against a lucky single split. The tiny ±0.20% std means the model is stable.

> **Important caveat:** these numbers are measured on **synthetic data** (§5). They demonstrate the model learned the patterns in the dataset - they are **not** a claim of real-world clinical accuracy.

4.6 Inference-time robustness (a key improvement)

Real users type only **2-4 symptoms**, not a full list. The training data was therefore generated with **sparse presentations** (each training row uses a random subset - roughly half to all - of a disease's symptoms). This

teaches the model to generalize to incomplete input instead of assuming near-complete symptom lists. Sparse inputs correctly produce a **spread of probabilities** (nuanced) rather than a false 100% on one class.

---

5. The Dataset

5.1 What it is

A **programmatically generated (synthetic) dataset** - `datasets/`: - `symptom_disease_dataset.csv` - **6,000 rows** × 135 columns (134 symptom columns of 0/1 + a `prognosis` label). **150 rows per disease**. - `disease_info.json` - for each of the 40 diseases: description, precautions, **home remedies**, **specialty** (department), severity, and core symptoms. - `symptoms_list.json` - the ordered list of 134 symptom identifiers.

5.2 How it's generated (`scripts/generate_dataset.py`)

Each disease has a curated set of **core symptoms** (commonly-known, general medical associations). For every training row we: 1. Randomly pick a **subset** of that disease's symptoms (~50-100% of them) - so the model sees varied, realistic, incomplete presentations. 2. Occasionally (15% chance) add **one unrelated "noise" symptom**, mimicking how real reports contain irrelevant details. 3. Label the row with the disease.

This forces the classifier to **generalize** (learn patterns) rather than do exact lookup.

5.3 Why synthetic data?

Reason	Explanation
No licensing/privacy issues	Real patient data is protected (HIPAA/GDPR) and hard to obtain legally for a student project.
Fully reproducible	Anyone can regenerate the exact dataset with one script - no giant download.
Controllable & balanced	We control the class balance and difficulty, ideal for demonstrating the ML pipeline.
Educational focus	The goal is to show the *engineering* (data -> model -> API -> UI), not to make clinical claims.

5.4 Dataset composition

- **40 diseases** across categories: respiratory (cold, flu, COVID-19, pneumonia, asthma...), gastrointestinal (GERD, ulcer, IBS...), infectious (dengue, malaria, typhoid...), chronic (hypertension, diabetes, thyroid...), dermatological, mental health, and more.
- **134 symptoms**.
- **Severity split:** 9 Low · 22 Medium · 9 High.
- **20 medical specialties** mapped (for the hospital module).

5.5 Honest limitation

Because the symptom-disease links are **general knowledge encoded by hand**, the dataset is **not clinically validated**. A real deployment would require a peer-reviewed, doctor-labeled dataset. This is stated openly throughout the app.

---

6. Module 2 - AI Chatbot (Rule-Based NLP)

6.1 What it does

Lets users describe symptoms in **plain English** ("I have a fever and a sore throat, but no cough"). It extracts symptoms, asks follow-up questions when it has too little information, detects emergencies, and - once it has enough signal - calls the same disease-prediction model.

6.2 The approach - rule-based NLP, no LLM

`chatbot_service.py` uses: - A **synonym dictionary** mapping natural phrases -> symptom IDs (e.g. "throwing up", "puking" -> `vomiting`). - **Clause-aware negation handling**: it splits text on punctuation and conjunctions, then checks a small window before each matched phrase for negation cues ("no", "not", "don't", "without"...). So *"fever but no vomiting"* correctly records **fever** and **drops vomiting** - a real correctness win. - **Emergency detection** (chest pain, difficulty breathing...) -> immediate "seek emergency care" response. - A **minimum-symptom threshold** (needs ≥3 symptoms) before predicting, with natural **follow-up questions** otherwise.

6.3 Why rule-based instead of a Large Language Model (LLM)?

Reason	Explanation
No API key / no cost	Runs 100% offline and free. An LLM (GPT/Claude) needs a paid API key and network access.
Deterministic & safe	For a medical-adjacent tool, predictable behavior matters. An LLM can *hallucinate* conditions or advice.
Privacy	No user health text is sent to a third-party service.
Transparent	Every extraction is traceable to a rule - easy to audit and explain.

The code is intentionally structured with a **clear swap-in point**: if an `OPENAI_API_KEY`/`ANTHROPIC_API_KEY` were provided, the extraction/generation functions could be replaced by an LLM **without changing the API or frontend**.

---

7. Module 3 - Image Detection (Heuristic, NOT a CNN)

> [OK] **This is the current, active implementation in the project.** The image detector uses a **computer-vision heuristic**, **not** a trained Convolutional Neural Network. (A CNN training pipeline

> exists in the codebase as an *optional, unused* path - see §7.4 - but **no CNN model is included or active**.)

7.1 What it does

The user uploads a skin photo. The service extracts visual features and scores them against candidate classes (Eczema, Psoriasis, Fungal Infection, Acne, Likely Normal), returning a **top-3 with confidences** plus **image-quality warnings**.

7.2 How the heuristic works (`image_service.py`)

Using only **NumPy + Pillow**, the image is resized and analysed for dermatology-relevant features: - **Erythema / redness** - how much red dominates green+blue (inflammation proxy). - **Texture / scaliness** - gradient (edge) energy (proxy for scaly plaques). - **Spottiness** - fraction of strong-gradient pixels (small bumps/spots). - **Pigmentation / darkness** and **brown-patch detection** (HSV hue range). - **Colour heterogeneity** - variance across the image.

These features are combined with **hand-tuned, transparent weights** per class, then passed through a **softmax** to produce probabilities. Two safety features: - **Image-quality gating**: detects too-dark, overexposed, blurry, or **non-skin** photos and **damps confidence toward uniform**, so it doesn't give a confident answer on an unusable image. - **Determinism**: the same image always yields the same result.

7.3 Why a heuristic and not a CNN?

Reason	Explanation
No labeled medical image dataset	A real skin-disease CNN needs a licensed, doctor-labeled dataset (e.g. HAM10000, ~2.6 GB). None is bundled.
No GPU / heavy framework required	A CNN needs TensorFlow/PyTorch and ideally a GPU. The heuristic runs anywhere with just NumPy + Pillow.
End-to-end contract works	The heuristic lets the **full pipeline** (upload -> analyse -> store -> display) work honestly for a demo, without pretending to diagnose.
Transparent & honest	Every score traces to a measurable feature. The UI clearly labels it as a **feature-based heuristic, not a medically validated model**.

7.4 The honest limitation

A colour/texture heuristic cannot medically diagnose skin disease. Two different conditions can look identical in colour, and the same condition looks different across skin tones, lighting, and cameras. This is a limitation of the **approach**, not a bug - the app states this plainly. Real accuracy requires a CNN trained on labeled data. The codebase includes an **optional, non-active** `scripts/train_image_model.py` (MobileNetV2 transfer-learning) and the service will *auto-use* a trained model **if one is ever dropped in** - but as shipped, **the heuristic is what runs.**

---

8. Module 4 - Recommendations, Risk & Hospitals

8.1 Recommendation engine

Not ML - a **curated knowledge base** (`disease_info.json`). For each condition: a plain-English description, precautions, and **home remedies** (supportive comfort measures, with emergency framing for serious conditions). This is more appropriate than ML because the content must be **accurate and controlled**, not generated.

8.2 Risk assessment

A **rules-based** combination of the model's **confidence** and the disease's baseline **severity** -> Low / Medium / High. Example: a High-severity condition with moderate confidence is flagged **High**, prompting urgent-care guidance.

8.3 Hospital locator (disease-aware)

Each disease is mapped to a **medical specialty** (Migraine->Neurology, Hypertension->Cardiology, Appendicitis->Emergency, ...). After a prediction, the app recommends the right **department** and produces **real Google Maps search links** for that kind of facility near the user's city. - The hospital **names are illustrative** (no live Places API key), but the **Maps links are genuinely useful** searches. Adding a `GOOGLE_MAPS_API_KEY` swaps in live results without code changes.

---

9. Supporting Modules (Auth, Dashboard, Reports)

- **Auth (JWT):** register/login issues a signed JWT; protected endpoints require it. Passwords are **bcrypt-hashed**. Stateless = scalable.
- **Dashboard:** aggregates the user's prediction history into charts (disease frequency, risk breakdown, confidence trend) via **Recharts**.
- **History & Reports:** every prediction is stored (SQLAlchemy); reports can be generated and downloaded as `.txt`.

---

10. Why These Choices Over Alternatives (Summary)

Layer	We chose	Over	Because
Frontend framework	React + Vite	Angular / CRA / Next	Fast, light, SPA-appropriate
Styling	Tailwind	Bootstrap / plain CSS	Consistent, fast, no context-switch
Backend	FastAPI	Flask / Django	Async, auto-docs, built-in validation

Auth	JWT	Server sessions	Stateless, scalable
ML model	Random Forest	Deep learning / LogReg / SVM	Best for small clean tabular data; captures interactions; no GPU
Chatbot	Rule-based NLP	LLM	Free, offline, deterministic, private
Image detection	**CV heuristic**	**CNN**	**No labeled dataset/GPU; honest demo of the full pipeline**
Recommendations	Curated KB	Generative AI	Accuracy & control over medical content
Database	SQLite -> PostgreSQL	MongoDB	Relational data; easy dev->prod switch

---

11. Limitations & Honesty

1. **Educational project, not medical software** - never for real health decisions.
2. **Synthetic dataset** - demonstrates the pipeline, not clinical validity.
3. **Image detector is a heuristic** - reads visual features, cannot diagnose.
4. **Hospital names are mock** (Maps links are real).
5. **No automated test suite** - verified manually.
6. `SECRET_KEY` must be set to a real secret before any deployment.

These limitations are surfaced **in the app's own disclaimers** - honesty about capability is a deliberate design principle here.

---

12. How to Run & Retrain

Run (local dev):

```
# Backend
cd backend
venv\Scripts\activate                # Windows
uvicorn app.main:app --reload --port 8000

# Frontend (second terminal)
cd frontend
npm run dev                          # http://localhost:5173
```

Regenerate data & retrain the ML model:

```
python scripts/generate_dataset.py    # rebuild the synthetic dataset
python scripts/train_model.py         # retrain + re-evaluate the Random Forest
```

Docker (both services):

```
docker-compose up --build
```

CareBot AI - built to demonstrate a complete, honest AI application: data generation -> model training -> API -> authentication -> interactive UI, with each technology chosen for a clear, defensible reason.